

Task 14 — : Build a Secure Application File Storage System

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-09-05

1. Approach and Architecture

The main goal of this task was to design a system where an application running on EC2 can securely store files in S3 without exposing long-term AWS credentials inside the application code. The development team wanted to place Access Keys and Secret Keys directly inside the application configuration, which is not a secure practice.

To solve this, I used IAM Roles instead of hardcoded credentials. I created an IAM role with the necessary S3 permissions and attached it to the EC2 instance. The Python application uses the boto3 library, which automatically fetches temporary credentials from the EC2 instance metadata. This ensures that credentials are never stored in the source code, are short-lived, and are automatically rotated by AWS.

The architecture is simple: an EC2 instance with an attached IAM role communicates with an S3 bucket to upload, list, and download files. The IAM role defines exactly what actions the application can perform on the bucket.

2. Implementation Steps

Step 1 – Understand the Application's File-Upload Requirement:

I identified that the application needs to upload, list, and download files from S3. Users upload documents, and the application must store them securely.

Step 2 – Create an S3 Bucket for Application Files: I created a private S3 bucket named task14-secure-bucket in the ap-south-1 region. I made sure that Block Public Access was enabled to keep the bucket private.

Step 3 – Define Exactly What the Application Needs to Do with the Bucket: I defined the specific actions the application requires: upload files, download files, and list files. I also noted that the application should not have permission to delete the bucket or manage S3 settings.

Step 4 – Create an Appropriate IAM Role/Policy: I created an IAM policy called EC2S3AccessPolicy that allows s3:PutObject, s3:GetObject, and s3:ListBucket on the specific bucket. Then I created an IAM role called EC2S3AccessRole with a trust policy allowing EC2 to assume the role, and attached the policy to it.

Step 5 – Attach the Role to the EC2 Instance: I launched an EC2 instance named task14-secure-app and attached the IAM role under the "IAM instance profile" section. This ensures the instance automatically gets the required permissions.

Step 6 – Configure the Application to Access S3 Using the Instance's AWS Permissions: I wrote a Python script using the boto3 library. The script uses the IAM role attached to the EC2 instance to authenticate with S3. No credentials are hardcoded in the script.

Step 7 – Ensure Long-Term Credentials Are Not Stored in Source Code: I verified that the Python script does not contain any Access Key ID or Secret Access Key. The script relies entirely on the IAM role for authentication.

Step 8 – Test Permitted Application Operations: I ran the Python script to upload a test file, list files, and download the file. All operations completed successfully.

Step 9 – Test Operations Outside the Application's Intended Permissions: I tried to delete the bucket and create a new bucket using the CLI. Both operations returned AccessDenied errors, confirming that the permissions were correctly restricted.

Step 10 – Configure Appropriate Object Organization: I stored files directly in the bucket root for simplicity. In a production environment, I would use prefixes to organize files by date or user.

Step 11 – Review Bucket Access Settings: I confirmed that Block Public Access was enabled and that the bucket policy only allowed access from the IAM role.

Step 12 – Document the Security Model: I documented the security model in the README and this report, explaining how IAM roles provide secure access.

Step 13 – Explain Why the Selected Approach Is Safer: I explained that IAM roles eliminate the risk of credential leakage, provide temporary credentials, and are automatically rotated by AWS.

Step 14 – Identify Additional Protections Required in Production: I identified additional protections like MFA Delete, S3 encryption, CloudTrail, and lifecycle policies.

Step 15 – Demonstrate the Complete Upload and Retrieval Workflow: I ran the Python script to demonstrate uploading, listing, and downloading files from S3.

3. Key AWS Concepts Learned

- **IAM Roles:** I learned how to create and use IAM roles to grant permissions to AWS services like EC2 without using long-term credentials.
- **Instance Metadata:** I understood how EC2 instances can retrieve temporary credentials from instance metadata using the IAM role.
- **Least Privilege:** I practiced granting only the minimum permissions required for the application to function.
- **boto3 Library:** I gained experience using the boto3 library in Python to interact with AWS services.
- **Temporary Credentials:** I learned that credentials provided by IAM roles are temporary and automatically rotated, which improves security.

- **Security Best Practices:** I understood the importance of avoiding hardcoded credentials in source code.
-

4. Challenges Faced

- **Understanding IAM Roles:** Initially, I was confused about how EC2 instances can assume IAM roles. I learned that the instance automatically gets temporary credentials from the metadata service.
 - **CLI Profile Mismatch:** When testing the Python script locally, I got AccessDenied errors because my default CLI profile was sns-user, which did not have S3 permissions. I had to use the --profile s3-admin flag or configure the script to use the correct profile.
 - **EC2 SSH Connectivity:** I faced issues connecting to the EC2 instance via SSH because the Security Group did not allow inbound SSH traffic. I fixed it by adding an inbound rule for SSH.
 - **Testing Denied Operations:** I had to carefully test denied operations using the CLI to confirm that the IAM role correctly restricted actions like deleting the bucket.
-

5. Solutions Implemented

- **IAM Role Setup:** I created an IAM role with a trust policy for EC2 and attached a custom policy that allowed only the required S3 actions on the specific bucket.
- **EC2 Configuration:** I launched an EC2 instance and attached the IAM role during the launch process. This ensured the instance automatically had the necessary permissions.
- **Python Script:** I wrote a Python script that uses boto3 to upload, list, and download files. The script does not contain any hardcoded credentials and relies on the IAM role.
- **Profile Management:** For local testing, I used boto3.Session(profile_name='s3-admin') to ensure the script used the correct profile with the necessary permissions.

- **Security Group Fix:** I added SSH and HTTP inbound rules to the Security Group to allow connectivity and testing.
-

6. Security Considerations

- **No Hardcoded Credentials:** The Python script does not contain any Access Key ID or Secret Access Key. All authentication is handled by the IAM role.
 - **Temporary Credentials:** The IAM role provides temporary credentials that are automatically rotated by AWS. This reduces the risk of credential leakage.
 - **Least Privilege:** The IAM policy only allows s3:PutObject, s3:GetObject, and s3:ListBucket on the specific bucket. No other actions are allowed.
 - **Private Bucket:** The S3 bucket has Block Public Access enabled, ensuring that files are not publicly accessible.
 - **Encryption:** The bucket uses default server-side encryption (SSE-S3) to protect data at rest.
 - **MFA Recommendation:** I recommended enabling MFA for IAM users to add an extra layer of security.
-

7. Results and Observations

Permitted Operations:

I successfully uploaded a test file to the S3 bucket using the Python script. The script also listed the files in the bucket and downloaded the file back to the local machine. All operations completed without errors.

Denied Operations:

When I tried to delete the bucket or create a new bucket using the CLI, I received AccessDenied errors. This confirmed that the IAM role correctly restricted the application's permissions.

Observations:

The IAM role approach is much more secure than hardcoding credentials. The application was able to interact with S3 without any credentials in the code, and the permissions were correctly enforced. The use of temporary credentials eliminates the need to rotate access keys manually.

8. Possible Improvements for Production

If this system were deployed in a production environment, I would recommend the following improvements:

- **S3 Server-Side Encryption:** Ensure all objects are encrypted at rest using SSE-S3 or KMS.
- **MFA Delete:** Enable MFA Delete on the bucket to prevent accidental permanent deletions.
- **CloudTrail Logging:** Enable CloudTrail to log all S3 and IAM API calls for auditing and compliance.
- **CloudFront + OAI:** If the application serves files publicly, use CloudFront with Origin Access Identity to restrict bucket access to CloudFront only.
- **Lifecycle Policies:** Use lifecycle policies to automatically transition older objects to cheaper storage tiers like Glacier.
- **Secrets Manager:** If the application needs third-party API keys, store them in AWS Secrets Manager instead of the application code.
- **Monitoring:** Set up CloudWatch alarms to monitor bucket activity and alert on unusual access patterns.

Conclusion

This task successfully demonstrated how to build a secure file storage system using EC2 and S3 with IAM roles. By eliminating hardcoded credentials, the system follows AWS best practices and significantly reduces the risk of credential leakage. The IAM role approach provides

temporary, automatically rotated credentials and enforces the principle of least privilege.